Chapter 1

Goal - find solution in optimal / linear complexity, $O(n \log^{\alpha} n)$

Idea behind hierarchical matrices (H-matrices):

a) decomposition of a matrix into rectangular blocks
b) approximation of every non-diagonal block as $\underbrace{\text{rank-}k \text{ matrix}}_{\text{number of linearly independent columns in } A}$

For example, $A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$

$\text{rank}(A) = 1$ } as second row = 2 × first row ⇒ not independent

$$A = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

$\text{rank}(A) = 2$

Block partition

$P_I = \{I_1, \ldots, I_p\} \subset \mathcal{P}(I) \setminus \{\phi\}$ is called a partition of I if.

a) $I_i \cap I_j = \phi \;\; \forall\, i \neq j$
b) $I = \bigcup_{j=1}^{p} I_j$

For example, $I = \{1, 2, 3, 4\}$

$I_1 = \{1, 2\}$, $I_2 = \{3, 4\}$

Partition, $P_I = \{I_1, I_2\} = \{\{1, 2\}, \{3, 4\}\}$

From condition a), $I_1 \cap I_2 = \phi$ (no overlap / disjoint)

b), $I = I_1 \cup I_2$ (complete)

For two index sets, I and J, $P_{I \times J} = \{b_1, \ldots, b_p\} \subset \mathcal{P}(I \times J) \setminus \{\phi\}$ is a block partition if

a) no overlap (disjoint)
b) complete
c) product structure → each block is a rectangle

Chapter 2

For rank-k matrices,

1) $0 \leq \text{rank}(M) \leq \min(n, m)$ - rank can't exceed no. of rows / cols
2) $\text{rank}(AB) \leq \min(\text{rank}(A), \text{rank}(B))$
3) $\text{rank}(A + B) \leq \text{rank}(A) + \text{rank}(B)$

Express rank-k matrix as $M = AB^T$ then $M = \underbrace{\sum_{i=1}^{k} a_i b_i^T}$

↳ each $a_i b_i^T$ is a rank-1 block

For example, $A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ $B = \begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix}$

Using the formula, $M = a_1 b_1^T + a_2 b_2^T$

$$a_1 b_1^T = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \begin{bmatrix} 2 & 4 \end{bmatrix} = \begin{bmatrix} 2 & 4 \\ 0 & 0 \end{bmatrix}$$

$$a_2 b_2^T = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \begin{bmatrix} 3 & 5 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 3 & 5 \end{bmatrix}$$

$$a_1 b_1^T + a_2 b_2^T = \begin{bmatrix} 2 & 4 \\ 3 & 5 \end{bmatrix} = M$$

Storage of $M \in R(k, n, m) = k(n+m)$

If $n = m = 1000$ and $k = 10$, $\longrightarrow$ full matrix $= n \times m = 1000000$

Rank-10 form $= 10(1000+1000)$

Operations with rank-k matrices

a) Matrix vector multiplication: $Mx = (A.B^T).x = A(B^T.x)$

cost for $y = B^T.x = k(2m-1)$

cost for $A.y = n(2k-1)$

Total cost $= 2k(m+n) - k - n \ll O(mn)$

b) Matrix matrix addition: $M' = A'B'^T$

$M'' = A''B''^T$

$$M' + M'' = \begin{bmatrix} A' & A'' \end{bmatrix} \begin{bmatrix} B'^T \\ B''^T \end{bmatrix} = AB^T \in R(k'+k'', n, m)$$

costs no operations, increases rank

c) Matrix matrix multiplication: $M' = A'B'^T \in R(k', n, m)$

$M'' = A''B''^T \in R(k'', m, l)$

$M'M'' = (A'B'^TA'')B''^T \in R(k'', n, l)$ (or)

$M'M'' = A'(B'^TA''B''^T) \in R(k', n, l)$

$(A'B'^TA'') = A'(B'^TA'')$ which costs

$N = 2k'k''(n+m) - k''(k'+n)$

$(B'^TA''B''^T) = (B'^TA'')B''^T$ which costs

$N = 2k'k''(m+l) - k'(k''+l)$

For $n = m = l$ and $k = k' = k''$,

$M'M'' \in R(k, n, n)$ with cost

$N = 2k^2(2n) - k(k+n)$

$N = 4k^2n - k^2 - kn$ costs $O(n)$

↓

linear cost and not n^3

For example, $n = 3$, $m = 4$, $l = 2$, $k' = 2$, $k'' = 2$

$A' = 3 \times 2$, $B'^T = 2 \times 4$, $A'' = 4 \times 2$, $B''^T = 2 \times 2$

If $M'M'' = (A'(B'^T A''))B''^T$

$$\left.\begin{aligned} B'^T A'' &= (2 \times 4) \times (4 \times 2) = 2 \times 2 \\ A'(2 \times 2) &= (3 \times 2) \times (2 \times 2) = 3 \times 2 \\ (3 \times 2) B''^T &= (3 \times 2) \times (2 \times 2) = 3 \times 2 \end{aligned}\right\} \text{only multiplying small matrices } (k', k'')$$

If $A, B \in R(k) \Rightarrow A + B \in R(2k)$ but goal is $A + B \approx M \in R(k)$

<u>SVD</u>

Let $M = U \Sigma V^T$ where $U \in \mathbb{R}^{n \times n}$, $V \in \mathbb{R}^{m \times m}$ orthogonal i.e., $U^T U = I_n$, $V^T V = I_m$, $\Sigma \in \mathbb{R}^{n \times m}$ diagonal with nonnegative diagonal entries $\sigma_i = \Sigma_{ii}$, $\Sigma_{ij} = 0$ if $i \neq j$, $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$

We want to minimize, $\underbrace{\min_{\text{rank}(R) \leq k} \|M - R\|_2}_{\text{2-norm (largest singular value difference)}}$ and $\underbrace{\min_{\text{rank}(R) \leq k} \|M - R\|_F}_{\text{Frobenius form (sum of squared differences)}}$

If $R_k = U \Sigma_k V^T$ with $\underbrace{(\Sigma_k)_{ij} = \begin{cases} \sigma_i & : i = j \leq k \\ 0 & : \text{else} \end{cases}}_{\text{keep only top-k singular values, set rest to 0}}$

For example, $M = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$

$\Sigma = \text{diag}(3, 1)$

Best rank-1 approximation, $\sigma_1 = 3$ and $\sigma_2 = 0$

$$R_1 = U \Sigma_1 V^T = \begin{bmatrix} 3 & 0 \\ 0 & 0 \end{bmatrix}$$

$$\Sigma = \begin{pmatrix} \sigma_1 & & \\ & \ddots & \\ & & \sigma_{\min(n,m)} \\ & 0 & \end{pmatrix} \in \mathbb{R}^{n \times m}$$

$$\Sigma_k = \begin{pmatrix} \sigma_1 & & & \\ & \ddots & & \\ & & \sigma_k & \\ & & & 0 \\ & & & & \ddots \end{pmatrix} \in \mathbb{R}^{n \times m}$$

Reduced SVD

Only first k non-zero singular values matter so we cut away all zeroes

For example, $M = \begin{bmatrix} 3 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}$

From SVD, $U = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$, $\Sigma = \begin{bmatrix} 3 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}$, $V = I_2$

From $\underbrace{\text{reduced SVD}}_{\text{smaller, same result}}$, $U_k = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}$, $\Sigma = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$, $V = I_2$

QR decomposition

$M = QR$ where $Q \in \mathbb{R}^{n \times n}$ is orthogonal
$R \in \mathbb{R}^{n \times m}$ is upper triangular

Compressed QR

If tall matrix, $M = QR = Q_m R_m$ } cost $= 4nm^2$ operations
much cheaper than SVD, $O(nm^2 + m^3)$

Truncation from rank-l to rank-k

From $M = AB^T$, $A \in \mathbb{R}^{n \times l}$, $B \in \mathbb{R}^{m \times l}$, $\text{rank}(M) \le l$

Goal - rank-k approximation ($k < l$)

Step 1 - Compressed QR of A
$A = Q_A R_A$ where $Q_A \in \mathbb{R}^{n \times l}$, $R_A \in \mathbb{R}^{l \times l}$
cost : $O(nl^2)$

Step 2 - Compressed QR of B
$B = Q_B R_B$ where $Q_B \in \mathbb{R}^{m \times l}$, $R_B \in \mathbb{R}^{l \times l}$
cost : $O(ml^2)$

Step 3 - $M = Q_A (R_A R_B^T) Q_B$

$R_A R_B^T = \hat{U} \Sigma \hat{V}^T$ cost : $O(l^3)$

Step 4 - $M = (Q_A \hat{U}) \Sigma (\hat{V}^T Q_B)$
$\underbrace{M = U \Sigma V^T}_{\text{gives same singular values as } M \text{ but using smaller computations}}$ where $U = Q_A \hat{U}$ and $V = Q_B \hat{V}$

Step 5 - To make it a rank-k approximation, keep top k singular values

$$R_k = U_k \Sigma_k V_k^T$$

Efficient because, a) two QR factorisations on small matrices
b) one SVD

Total cost $\approx O((n+m)\ell^2 + \ell^3) \ll O(nm^2)$ for full SVD

Computation of compressed SVD, $M = U \Sigma V^T$ requires $6\ell^2(n+m) + 22\ell^3$ operations

If $n = 1000$, $m = 800$, $\ell = 50$, $k = 10$ then full SVD $\Rightarrow 1000 \times 800$ ops else

$QR(A) = 1000 \times 50$

$QR(B) = 800 \times 50$

$SVD(50 \times 50)$

total $= 22\ell^3 + 6\ell^2(n+m)$

≈ 2.7 mil ops $\ll$ SVD ops

Truncation

Keep the k largest singular values and discard the rest

$$T^R_{k \leftarrow \ell} : R(\ell, n, m) \to R(k, n, m)$$

Rank-k Matrix Addition

Two low rank matrices, $M' = A'B'^T \in R(k', n, m)$

$M'' = A''B''^T \in R(k'', n, m)$

The problem with $M' + M'' \to$ even if M' and M'' are low rank, their sum might have a larger rank,

$$\text{rank}(M' + M'') \leq k' + k''$$

$\Rightarrow$ lose low rank structure and memory advantage

The solution - $\underbrace{M' \oplus_k M'' = T^R_{k \leftarrow k'+k''}(M' + M'')}$

a) form the sum

b) truncate $T^R_{k \leftarrow k'+k''}$ to reduce rank $\leq k$

cost : at most $24k^2(n+m) + 176k^3$

Formatted Agglomeration

↳ explains merging / concatenating two low rank matrices

$M_1 = A_1 B_1^T \in R(k_1, n, m_1)$, $M_2 = A_2 B_2^T \in R(k_2, n, m_2)$

if $(M_1 \;\; M_2) \Rightarrow$ size of $n(m_1+m_2)$ and rank k_1+k_2 but we need $k < k_1+k_2$

$$(M_1 \;\; M_2) = (A_1 B_1^T \;\; A_2 B_2^T) = (A_1 B_1^T \; 0) + (0 \; A_2 B_2^T)$$

$$= A_1 (B_1^T \; 0) + A_2 (0 \; B_2^T)$$

$$= (A_1 \;\; A_2) \begin{pmatrix} B_1^T & 0 \\ 0 & B_2^T \end{pmatrix}$$

→ helps merging without extra overlap

Truncate : $T^R_{k \leftarrow k_1+k_2}\left((A_1 \;\; A_2) \begin{pmatrix} B_1^T & 0 \\ 0 & B_2^T \end{pmatrix} \right) \in R(k, n, m_1+m_2)$

Instead of formatted addition, instead of summing blocks we concatenate horizontally

cost of agglomeration < cost of formatted addition due to zero blocks

① QR decomposition of $(A_1 \;\; A_2)$:

$$(A_1 \;\; A_2) = Q_A R_A$$

② QR decomposition of $\begin{pmatrix} B_1 & 0 \\ 0 & B_2 \end{pmatrix}$:

$$\begin{pmatrix} B_1 & 0 \\ 0 & B_2 \end{pmatrix} = Q_B R_B$$

$$Q_B = \begin{pmatrix} Q_{B_1} & 0 \\ 0 & Q_{B_2} \end{pmatrix} \qquad R_B = \begin{pmatrix} R_{B_1} & 0 \\ 0 & R_{B_2} \end{pmatrix}$$

③ SVD of $R_A R_B^T$

④ $U = Q_A \hat{U}$, $V = Q_B \hat{V}$

Total cost: half of formatted addition due to zero blocks

$$A_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \quad B_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad M_1 = A_1 B_1^T = \begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$

$$A_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \quad B_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \quad M_2 = A_2 B_2^T = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}$$

$$\begin{pmatrix} M_1 & M_2 \end{pmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{pmatrix} A_1 & A_2 \end{pmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix} \quad \begin{pmatrix} B_1^T & 0 \\ 0 & B_2^T \end{pmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

<u>Step 1</u> - QR of $\begin{pmatrix} A_1 & A_2 \end{pmatrix}$

$$\begin{pmatrix} A_1 & A_2 \end{pmatrix} = Q_A R_A$$

$$Q_A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix} \quad R_A = I_2$$

<u>Step 2</u> - QR of block diagonal

$$\begin{pmatrix} B_1^T & 0 \\ 0 & B_2^T \end{pmatrix} = Q_B R_B$$

$$Q_B = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad R_B = I_2$$

<u>Step 3</u> - SVD of $R_A R_B^T$

$$R_A R_B^T = I_2$$

$$\hat{U} = I_2, \ \Sigma = I_2, \ \hat{V} = I_2$$

$\Rightarrow$ SVD on small $(k_1 + k_2) \times (k_1 + k_2)$ matrix and not $n(m_1 + m_2)$

<u>Step 4</u> - $U = Q_A \hat{U} = Q_A$

$V = Q_B \hat{V} = Q_B$

$\Rightarrow \begin{pmatrix} M_1 & M_2 \end{pmatrix} = U \Sigma V^T$ (SVD form)

If add or agglomerate more than two low rank matrices

→ pairwise truncation

→ instead of adding everything at once, do it step by step:

$$M = T^{R}_{k \leftarrow k_1 + k_2}\left(M_1 + T^{R}_{k \leftarrow k_3 + k_4}(M_3 + M_4)\right)$$

$$M = T^{R}_{k,\text{pairwise}}\left(\sum_{i=1}^{n} M_i\right) \Big\} \text{ much cheaper}$$

Chapter 3

Hierarchical matrices, H_p } p controls how many times the matrix is split into blocks

H_p is defined recursively → build a big matrix using smaller H_{p-1} ones

$$M = \begin{bmatrix} M_{11} & M_{12} \\ M_{21} & M_{22} \end{bmatrix}$$

If $p = 0$, $M \in \mathbb{R}^{1\times 1} \Rightarrow H_0 = \mathbb{R}^{1\times 1}$

If $p > 0$, M_{11} and $M_{22} \in H_{p-1}$ (top left and bottom right blocks are H matrices)
M_{12} and $M_{21} \in R(k, 2^{p-1}, 2^{p-1}) \Rightarrow$ off-diagonal blocks have low rank structure

When $k = 1$, $H_p = \begin{bmatrix} H_{p-1} & R_{p-1} \\ R_{p-1} & H_{p-1} \end{bmatrix}$ } each off diagonal block is a rank-1 matrix, $R_{p-1} = R(1, 2^{p-1}, 2^{p-1})$

Number of blocks, $N_{block}(p) = 3n - 2$ where $n = 2^p$

Storage costs

$S_R(p)$ = storage cost of $R_p = \underbrace{2^{p+1}}$

↓

rank 1 matrix of size $2^p \times 2^p$

needs 2^{p+1} storage units

For H_p matrix, $H_p = \begin{bmatrix} H_{p-1} & R_{p-1} \\ R_{p-1} & H_{p-1} \end{bmatrix}$

To store H_p, → two smaller H_{p-1} blocks
→ two R_{p-1} blocks

$$S_H(p) = 2 \cdot S_H(p-1) + 2 S_R(p-1)$$

$$\boxed{S_R(p-1) = 2^p \Rightarrow S_H(p) = 2 S_H(p-1) + 2^{p+1}}$$

$$\begin{aligned} S_H(p) &= 2 S_H(p-1) + 2^{p+1} \\ &= 2[2 S_H(p-2) + 2^p] + 2^{p+1} \\ &= 2^2 S_H(p-2) + 2 \cdot 2^p + 2^{p+1} \end{aligned}$$

$$\boxed{S_H(p) = 2^2 S_H(p-2) + 2^{p+2}}$$

$$\begin{aligned} S_H(p) &= 2^p S_H(0) + p \cdot 2^{p+1} \\ &= 2^p + p \cdot 2^{p+1} \quad (S_H(0) = 1) \end{aligned}$$

$$S_H(p) = n + 2n\log n \quad (n = 2^p) \Rightarrow O(n \log n)$$

Matrix vector multiplication

Step 1 - Multiplying an R_p matrix by a vector requires $3n-1$ operations

$$n = 2^p$$

Step 2 - Multiply H_p by a vector

$$H_p \cdot x = \begin{bmatrix} H_{p-1} x_1 + R_{p-1} x_2 \\ R_{p-1} x_1 + H_{p-1} x_2 \end{bmatrix} \rightarrow \text{two recursive multiplications with smaller } H_{p-1}$$

Step 3 - $N_{MV}(p) = 2N_{MV}(p-1) + 2(\text{cost of } R_{p-1})$

$$\text{cost of } R_{p-1} = 3\,n/2 - 1 \quad [n/2 = 2^{p-1}]$$

$$\Rightarrow N_{MV}(p) = 2N_{MV}(p-1) + 2\left(3 \cdot \frac{n}{2} - 1\right)$$

$$= 2N_{MV}(p-1) + 4n - 2$$

Step 4 - $N_{MV}(0) = 1$

Step 5 - $N_{MV}(p) = 2N_{MV}(p-1) + 4n - 2$

$$= 2\left(2N_{MV}(p-2) + 4 \cdot n/2 - 2\right) + 4n - 2$$

$$= 2^2\left(N_{MV}(p-2)\right) + 8n - 6$$

$$\Rightarrow N_{MV}(p) = 2^p N_{MV}(0) + p \cdot 4n - \sum_{i=1}^{p} 2^i$$

$$N_{MV}(p) = 4n \log_2 n - n + 2 \Rightarrow O(n \log n)$$

Matrix matrix addition

$$\begin{bmatrix} H & R \\ R & H \end{bmatrix} \oplus \begin{bmatrix} H & R \\ R & H \end{bmatrix} = \begin{bmatrix} H \oplus H & R \oplus R \\ \underbrace{R \oplus R}_{\text{formatted addition}} & H \oplus H \end{bmatrix}$$

$$N_{H \oplus H}(p) = 17n \log_2 n + 39n - 38 \Rightarrow O(n \log n)$$

Matrix matrix multiplication

$$\begin{bmatrix} H & R \\ R & H \end{bmatrix} \odot \begin{bmatrix} H & R \\ R & H \end{bmatrix} = \begin{bmatrix} HH \oplus RR & HR \oplus RH \\ RH \oplus HR & RR \oplus HH \end{bmatrix}$$

Case 1 - $R.R \in R$, cost $N_{RR}(p) = 3n - 1$

Case 2 - $H.R \in R$ or $R.H \in R$, cost $N_{HR}(p) = N_{RH}(p) = 4n\log n - n + 2$

Case 3 - $H.H \notin H$, $N_{HH}(p) \in O(n\log_2^2 n)$

Matrix inversion

$$M = \begin{pmatrix} M_{11} & M_{12} \\ M_{21} & M_{22} \end{pmatrix}$$

Schur complement, $S = M_{22} - M_{21} M_{11}^{-1} M_{12}$

$$M^{-1} = \begin{pmatrix} M_{11}^{-1} + M_{11}^{-1} M_{12} S^{-1} M_{21} M_{11}^{-1} & -M_{11}^{-1} M_{12} S^{-1} \\ -S^{-1} M_{21} M_{11}^{-1} & S^{-1} \end{pmatrix}$$

To invert M, ① one inverse of M_{11}
② S
③ inverse of S

In H-matrices, each block is stored in a hierarchical way, apply 2×2 trick recursively until the blocks are 1×1

$N_{inv}(p) = 12.5\, n\log_2^2 n + 27.5\, n\log_2 n - 69n + 70 \in O(n\log_2^2 n)$

↳ block tree of depth $O(\log n)$ and each level involves $O(\log n)$ work

LU decomposition

$$M = \begin{pmatrix} M_{11} & M_{12} \\ M_{21} & M_{22} \end{pmatrix} = \underbrace{\begin{pmatrix} L_{11} & 0 \\ L_{21} & L_{22} \end{pmatrix}}_{L} \underbrace{\begin{pmatrix} U_{11} & U_{12} \\ 0 & U_{22} \end{pmatrix}}_{U}$$

a) Factor top left, $M_{11} = L_{11} U_{11}$

b) compute off diagonals, $L_{11} U_{12} = M_{12}$ } lower triangular
$U_{12} = L_{11}^{-1} M_{12}$

$L_{21} U_{11} = M_{21}$ } upper triangular
$L_{21} = M_{21} U_{11}^{-1}$

c) Form and factor the Schur complement, $S = M_{22} - L_{21} U_{12}$
then recursively, $S = L_{22} U_{22}$

$$N_{LU}(p) = \frac{21}{4} n\log_2^2 n + \frac{61}{4} n\log_2 n - 37(n-1) \in O(n\log^2 n)$$

Tridiagonal Matrix

$$M = \begin{pmatrix} M_{11} & M_{12} \\ M_{21} & M_{22} \end{pmatrix}$$

In M, off diagonal blocks $\underbrace{M_{12}, M_{21}}_{\text{very low rank} \leq 1}$ each have at most one nonzero per row/column

Recurse of diagonal blocks, M_{11} and M_{22}

$\longrightarrow$ Inverse of tridiagonal matrix still has H-structure

A one dimensional integral equation model problem

$\hookrightarrow$ how H matrices approximate dense matrices

$$f(x) = \int_0^1 \underbrace{g(x,y)}_{\text{kernel}} \underbrace{u(y)}_{\text{unknown}} \, dy$$

Split the interval $[0,1]$ into n small subintervals,

$$0 = x_0 < x_1 < \dots < x_n = 1$$

For each subinterval, $\varphi_i(x) = \begin{cases} 1, & x \in [x_{i-1}, x_i] \\ 0, & \text{else} \end{cases}$

Approximate $u(x)$ as linear combination of the boxes,

$$u(x) \approx \sum_{j=1}^{n} \underbrace{z_j}_{\text{coefficients (unknown)}} \varphi_j(x)$$

$$f(x) = \int g(x,y) \left(\sum_{j=1}^{n} z_j \varphi_j(y) \right) dy = \sum_{j=1}^{n} z_j \int g(x,y) \varphi_j(y) \, dy$$

Multiply both sides by $\varphi_i(x)$ and integrate

$$\underbrace{\int \varphi_i(x) f(x) \, dx}_{b} = \underbrace{\sum_{j=1}^{n} z_j}_{z} \underbrace{\int \varphi_i(x) \int g(x,y) \varphi_i(y) \, dy \, dx}_{G}$$

$$Gz = b$$

$\downarrow$

has no zeros $\Rightarrow$ huge matrix $\Rightarrow$ compress G

Let $g(x,y) = -\log|x-y|$, when x and y are close $\Rightarrow$ g changes rapidly

x and y are far $\Rightarrow$ g changes smoothly

$\Rightarrow$ for far blocks of G, approximate $g(x,y)$ by Taylor expansion

For a fixed y,

$$g(x,y) \approx \tilde{g}(x,y) = \sum_{\nu=0}^{k-1} \frac{(x-x_t)^\nu}{\nu!} \frac{\partial^\nu g}{\partial x^\nu}(x_t, y)$$

x_t → center in internal of x

$$\tilde{g}(x,y) = \sum_{\nu=0}^{k-1} a_\nu(x)\, b_\nu(y)$$

For each block entry,

$$g_{ij} = \int_{x_{i-1}}^{x_i} \int_{y_{i-1}}^{y_i} g(x,y)\, dy\, dx$$

g_{ij} can be approximated by,

$$\tilde{g}_{ij} = \sum_{\nu=0}^{k-1} \int_{x_{i-1}}^{x_i} a_\nu(x)\, dx \int_{x_{j-1}}^{x_j} b_\nu(y)\, dy = \sum_{\nu=0}^{k-1} a_{i\nu}\, b_{j\nu}$$

$$G_m \simeq AB^T$$

For H-matrices, ① near the diagonal $(x=y)$ ⇒ kernel rapidly changes ⇒ keep full data

② far from diagonal ⇒ kernel is smooth ⇒ approximate low rank form AB^T

Efficiency of $O(n \log^2 n)$

Theorem 3.5 tells how big the approximation error is b/w the real $g(x,y)$ and Taylor approximated $\tilde{g}(x,y)$

Let $I_{left} = [x_{i_1}, x_{i_2}]$, $I_{right} = [x_{j_1}, x_{j_2}]$ where $i_2 < j_1$

We expand $g(x,y)$ in x around the mid point of I_{left},

$x_t = \frac{1}{2}(x_{i_1} + x_{i_2})$ using a Taylor polynomial of order k

$$|g(x,y) - \tilde{g}(x,y)| \le \log(1+\eta) \left(\frac{\eta}{1+\eta}\right)^{k-1}$$

$$\eta = \frac{\text{diam}(I_{left}) \to \text{length of } I_{left}}{2\,\text{dist}(I_{left}, I_{right}) \to \text{gap b/w two intervals}}$$

→ If intervals are close together, denominator is small ⇒ big error

well separated ⇒ η is small

For example, $\eta = \frac{1}{4h}$,

$$|g(x,y) - \tilde{g}(x,y)| \le \log\left(1 + \frac{1}{4h}\right)\left(\frac{1}{1+4h}\right)^{k-1} = f(h,k)$$

h → distance b/w intervals

k → Taylor order

For larger $h \Rightarrow$ intervals are apart, a small k gives tiny error $\Rightarrow$ low rank works

small $h \Rightarrow$ intervals close $\Rightarrow$ need large k to get same error $\Rightarrow$ poor approximation

Hence, apply low rank approximation where η is small $\Rightarrow$ where intervals are apart

$$\underbrace{\text{Close by blocks remain full (no compression)}}_{\substack{\text{H - matrix rule} \\ \text{(compress far blocks, store near blocks in full)}}}$$

$\text{diam}(I_{left}) \leq \eta_0 \, \text{dist}(I_{left}, I_{right})$ } size of a interval (block) must be small compared to its distance to another block

Chapter 4

Building a matrix G from a kernel function $g(x,y)$,

$$G_{ij} = g(x_i, y_i) \quad \text{or} \quad G_{ij} = \iint g(x,y)\,dx\,dy$$

$\downarrow$

defines how every pair of points interact

but problem is G is dense with n^2 entries

Separable functions

$g(x,y)$ written as sum of separable terms,

$$g(x,y) = \underbrace{\sum_{v=1}^{k} a_v(x)\, b_v(y)}_{\text{separation rank}}$$

$\downarrow$

separable / degenerate

$G = AB^T$ where separation rank of kernel = matrix rank

Interpolation

Use Lagrange interpolation polynomial,

$$p(x,y) = \sum_v g(\xi_v, y)\, \underbrace{L_v(x)}_{\text{lagrange basis polynomial}}$$

At the interpolation point $x = \xi_v$,

$$\underbrace{p(\xi_v, y)}_{\text{approximates } g} = g(\xi_v, y) \Big\} \; p(x,y) \text{ exactly matches } g(x,y) \text{ at those nodes}$$

The methodology

① Start with 1D Lagrange interpolation for $x \in [a,b]$: approximate $f(x)$ by a polynomial $P_f(x)$ that passes through known values $f(\xi_0), \ldots, f(\xi_m)$

$$P_f(x) = \sum_{v=0}^{m} f(\xi_v)\, L_v(x)$$

② Extend to higher dimensions: For 2D kernels, use tensor grids

$$\underbrace{p(x,y)}_{\text{separable with rank } k \,=\, \text{number of interpolation points}} = \sum_v \underbrace{g(\xi_v, y)}_{b_v(y)}\, \underbrace{L_v(x)}_{a_v(x)} \Big\} \text{ sum of separable terms}$$

$\Rightarrow$ a low rank approximation w/o using derivatives

One dimensional interpolation

Let $f : [a,b] \to \mathbb{R}$ and interpolation points $\xi_0, \dots, \xi_m$ then

$$\left.\begin{aligned} L_\nu(x) &= \prod_{\substack{\mu=0 \\ \mu \neq \nu}}^{m} \frac{x - \xi_\mu}{\xi_\nu - \xi_\mu} \end{aligned}\right\} \text{Lagrange basis}$$

$$p_f(x) = \underbrace{\sum_{\nu=0}^{m} f(\xi_\nu)\, L_\nu(x)}_{\substack{\text{polynomial of degree } m \\ \text{goes through the given points}}}$$

Error of Interpolation

Error at any $x \in [a,b]$ is:

$$f(x) - p_f(x) = \frac{1}{(m+1)!}\, \omega(x)\, f^{(m+1)}(\eta)$$

for some $\eta \in [a,b]$ where,

$$\omega(x) = \prod_{\nu=0}^{m} (x - \xi_\nu)$$

$$\|f - p_f\|_{\infty,[a,b]} \leq \frac{1}{(m+1)!} \underbrace{\|\omega\|_{\infty,[a,b]}}_{\leq (b-a)^{m+1}} \|f^{(m+1)}\|_{\infty,[a,b]}$$

Improving the error bound $\to$ choose Chebyshev points instead of equally spaced ones

$\Rightarrow \|\omega\|_{\infty,[a,b]}$ becomes much smaller $\Big\}$ smaller interpolation error

$\Rightarrow$ Chebyshev nodes are often used in H-matrix implementations $\Rightarrow$ leads to low rank blocks in G

Zeros of $\underbrace{\text{Chebyshev polynomials}}_{\substack{\text{family of oscillating polynomials} \\ \text{on } [-1,1]}}$ make $\|\omega\|_\infty$ small

Trigonometric identity, T_n is a cosine wave stretched to fit $[-1,1]$. The zeros of T_n are $\underbrace{\text{Chebyshev nodes}}_{\text{used as interpolation points}}$ which are denser near ± 1

Stability constant, C_{stab} quantifies amplification of error

If $C_{stab} = 1$, perfectly stable
$C_{stab} \gg 1$, unstable

C_L, lebesgue constant measures how much the interpolation process can magnify data values $\Rightarrow$ measure of numerical stability

$$C_{stab} = \frac{2}{\pi} \ln(m+1) + 1$$

} if m (interpolation degree) increases a lot, amplification of error grows slowly (as log)

for equal spaced nodes, $C_{stab} \uparrow\uparrow$ which makes interpolation unstable for large m

The interpolation error is at most $(1 + C_{stab})$ times worse than the best achievable polynomial approximation

$$\boxed{\begin{array}{l} \text{Lemma 4.6} \rightarrow \text{controls accuracy} \\ \text{Lemma 4.8 + 4.9} \rightarrow \text{stability} \end{array}}$$